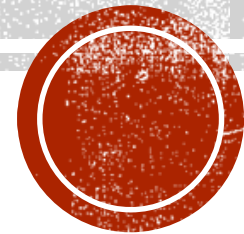


RECONFIGURABLE COMPUTING

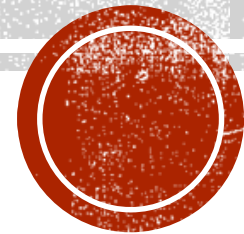
CS G553: Reconfigurable computing

Dr. Meetha V Shenoy



ARCHITECTURAL SYNTHESIS (HIGH LEVEL SYNTHESIS)

TEMPORAL PARTITIONING & SCHEDULING



Partitioning

A design implementation is often too big to allow an implementation on a single FPGA.

Possible solutions:

Spatial
Partitioning

The design is partitioned into many FPGAs.

Each partition block is implemented in one single FPGA.

All the FPGAs are used simultaneously.



Partitioning

A design implementation is often too big to allow an implementation on a single FPGA.

Possible solutions:

Temporal
Partitioning

The design is partitioned into blocks

Each of which will be executed on one FPGA at a given time



Spatial Partitioning

Partitioning Constraints: Each FPGA is characterized by:

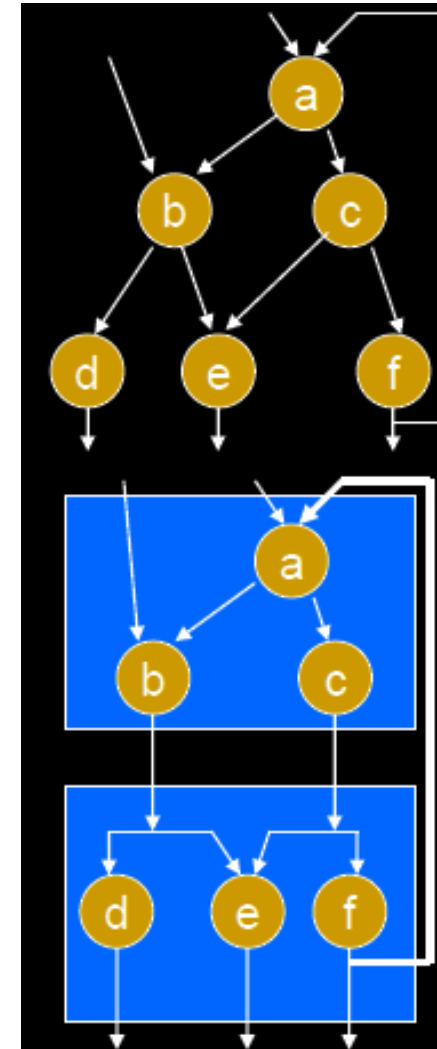
The size, i.e., the number of LUTs, FFs available

The terminals, i.e., the number of I/O pins available on the device

A partition is valid iff: for a block B produced by the partition, we have:

$S(B) \leq S(\text{device})$ where $S(X)$ = size of X

$T(B) \leq T(\text{device})$ where $T(X)$ = # terminals of X



Spatial Partitioning

Objectives: The following objectives are possible:

Minimize the number of cut nets

Minimize the number of produced blocks

Minimize the delay

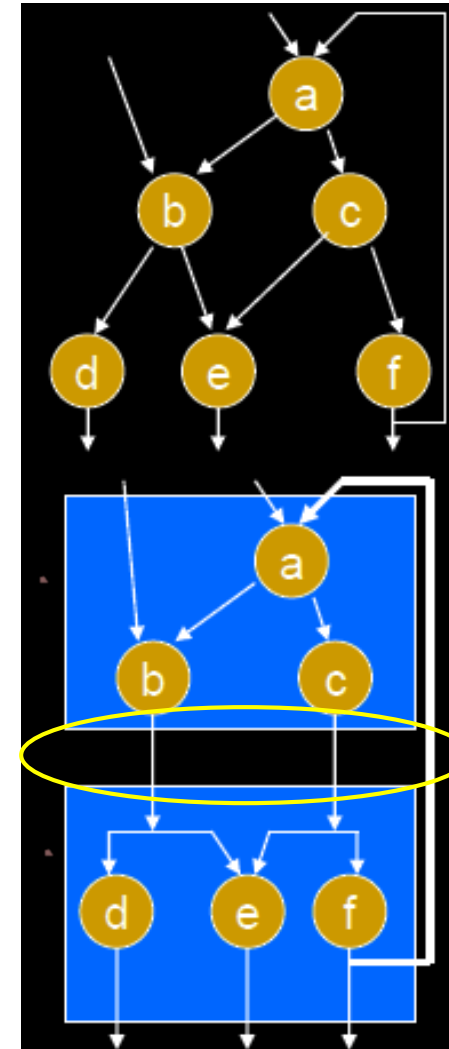
Difficult problem due to all the constraints which are not always compatible.

Solution approaches:

Use of heuristics for automatic partitioning

Manual intervention

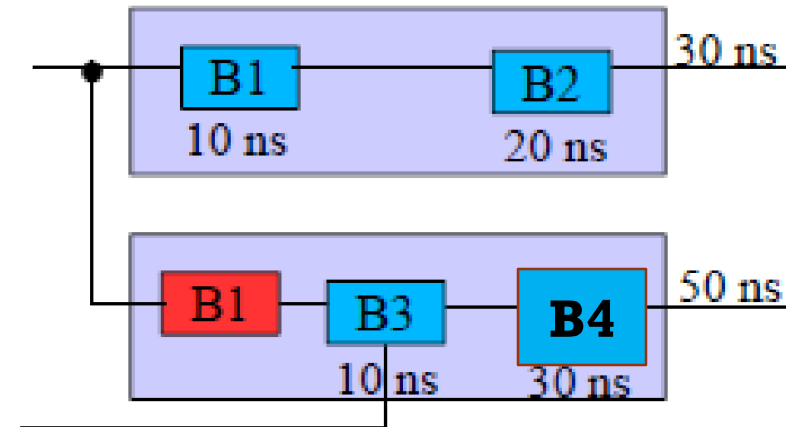
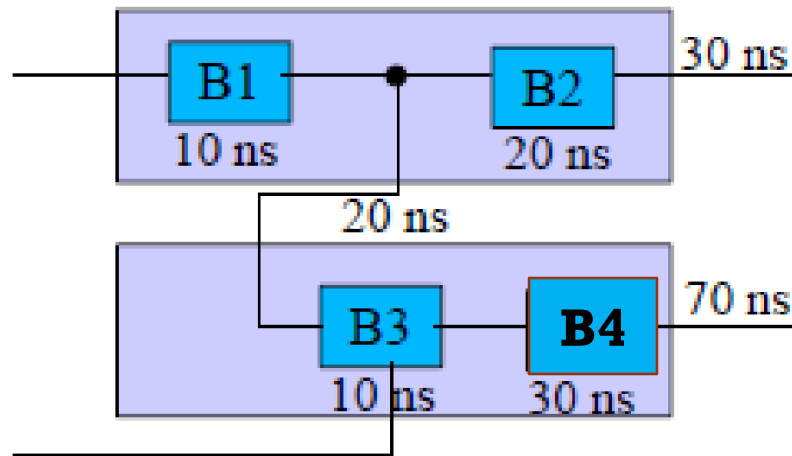
RC@Meetha V Shenoy



Spatial Partitioning

Timing block Replication

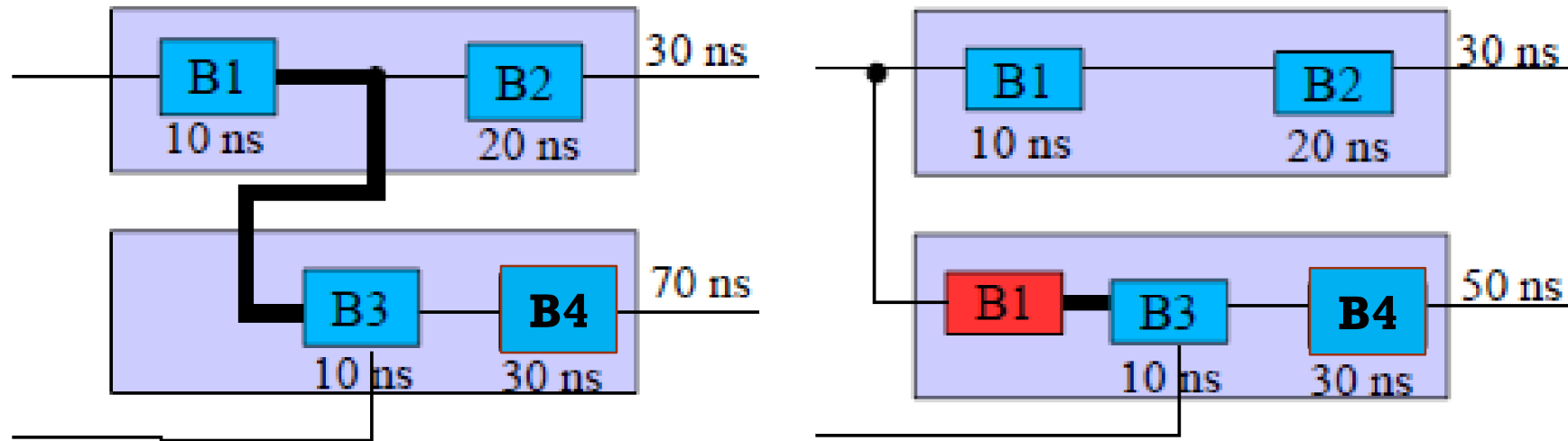
**Critical
Path
Optimization**



Spatial Partitioning

Timing block Replication

Reduce the Number of I/O Pins



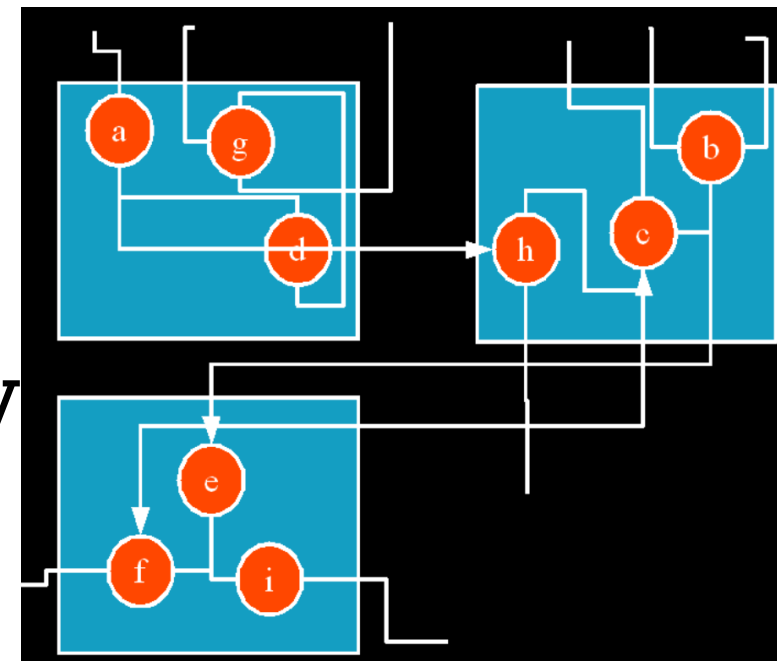
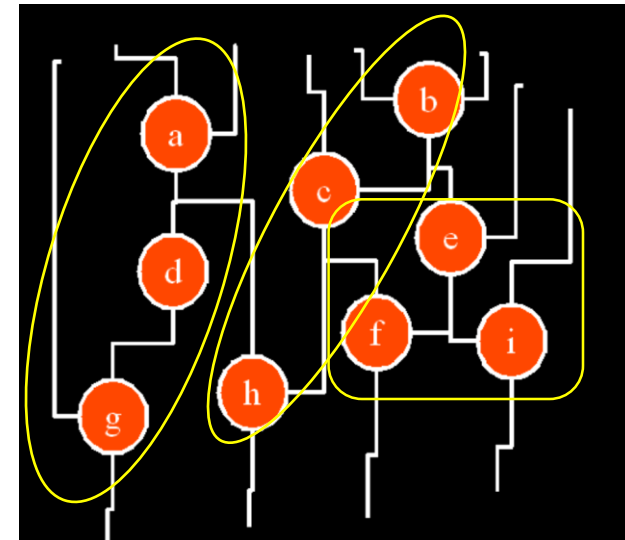
Temporal Partitioning

Resources on the device are not allocated to only one operator

But to a set of operators that must be placed at the same time and removed.

Application partitioned in to sets of operators

Partitions will then be successively implemented at different time

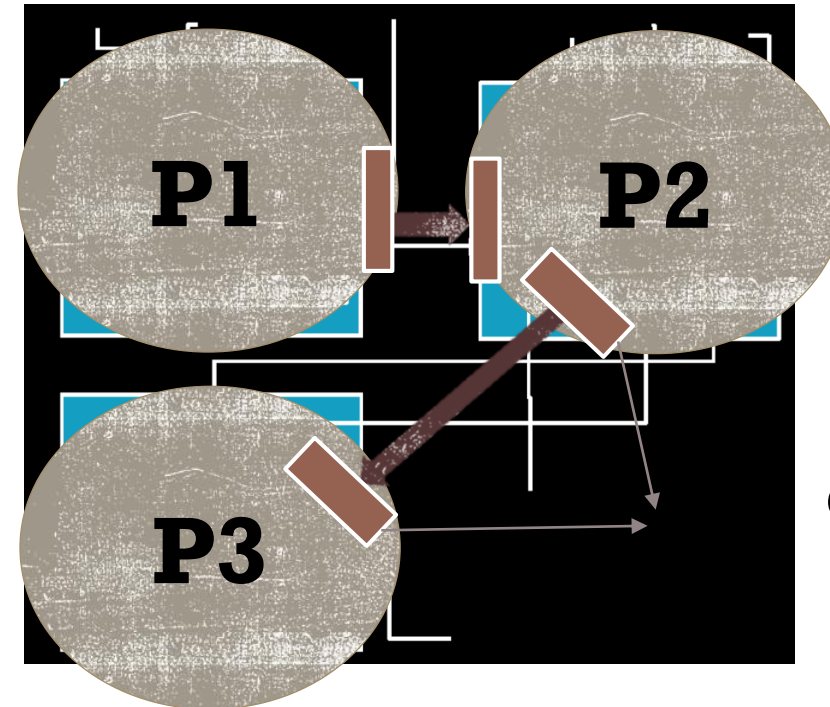
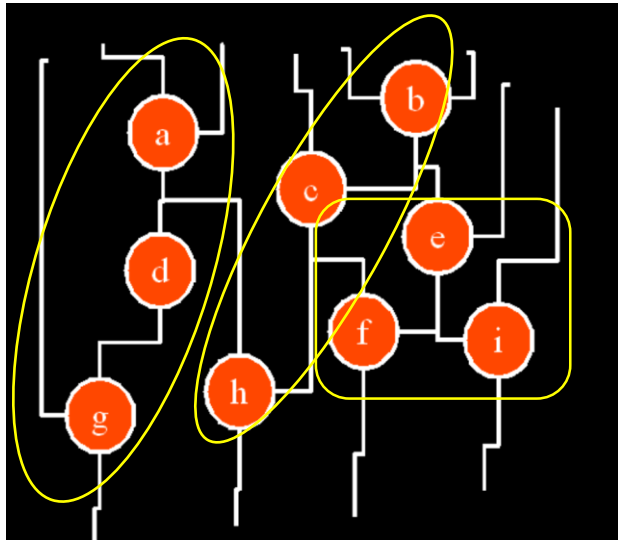


Configuration Graph

A configuration graph is a graph in which:

Nodes are partitions or bitstreams

Edges reflect the precedence constraints in a given DFG



Inter
Configuration
Registers



Configuration Graph

Given a DFG $G = (V, E)$ and
a temporal partitioning $P = \{P_1, \dots, P_n\}$ of G

We define configuration graph of G relative to the P , with
notation $\Gamma(G/P)$

$\Gamma(G/P) = (P, E^P)$ in which the nodes are partitions in P .

An edge

$$e^P = (P_i, P_j) \in E^P \Leftrightarrow \exists e = (v_i, v_j) \in E \\ \text{with } v_i \in P_i \text{ and } v_j \in P_j.$$



Temporal Partitioning

Whenever a new partition is downloaded, the partition that was running is destroyed.

Communication through inter-configuration registers (or communication memory)

May sit in main memory

May sit at the boundary of the device to hold the input and output values

Configuration sequence is controlled by the host processor



Temporal Partitioning

Steps (for P_i and P_j , ($P_i \leq P_j$)) :

Configuration for P_i is first downloaded into the device.

Executes

P_i copies all the data it needs to send to other partitions into the communication memory.

The device is reconfigured to implement the partition P_j

Accesses the communication memory and collect the data.



Temporal Partitioning

Objectives for optimization:

interconnections:

Minimizes exchanged data and temporary storage

produced blocks (partitions):

Reduces the number of reconfigurations

overall computation delay:

Depends on partition run time, processor used for configuration and speed of data exchange

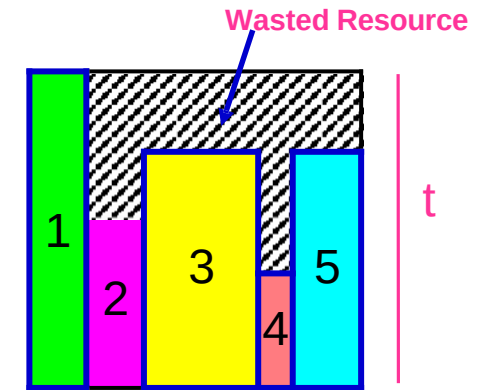
Similarity between consecutive partitions

Partial Reconfiguration

overall amount of wasted resources on the chip.



RESOURCE WASTAGE MINIMIZATION



□ Wasted resource avoidance:

- A component is placed on the chip only when its computation is required and remains on the device only for time it is active.
 - Idle components can be replaced by new ones.

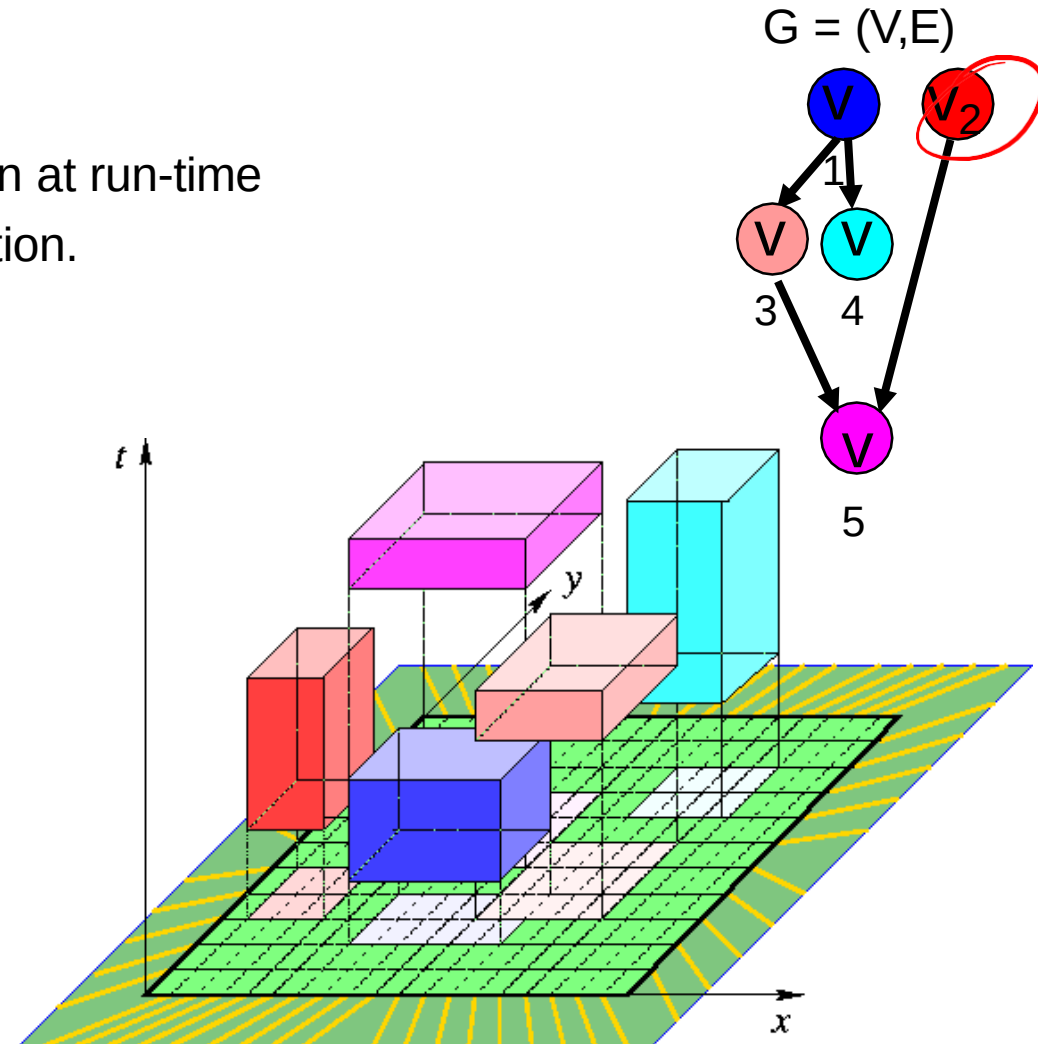


TEMPORAL PLACEMENT

□ Temporal placement:

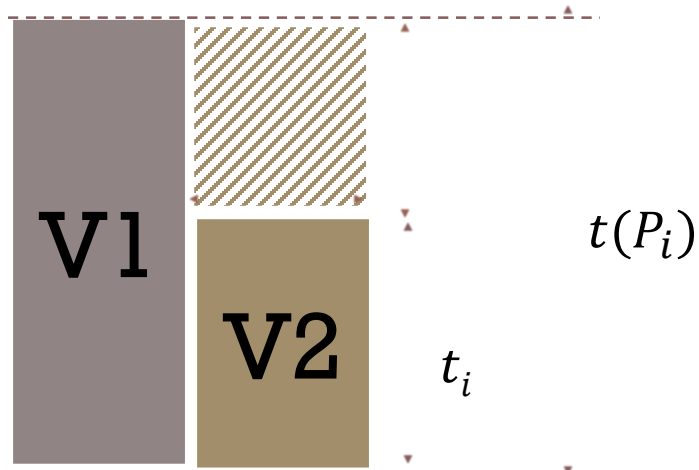
- Time-dependent placement
- Management of task execution at run-time
- Z axis: life time of a configuration.

➤ A horizontal cut:
configuration at a given
time



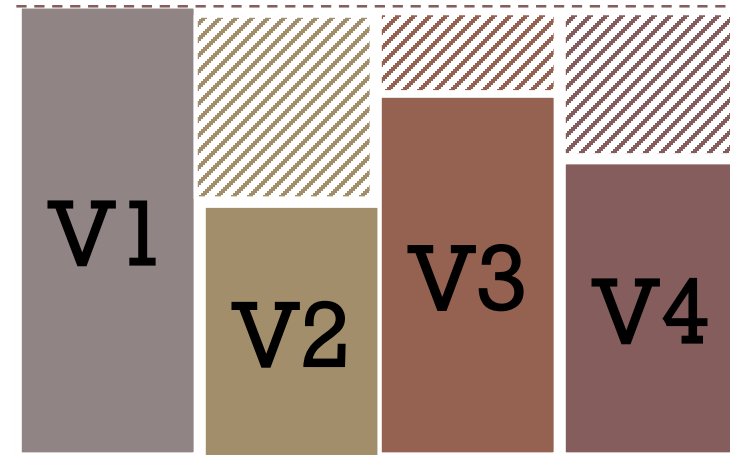
Wasted Resources

Run time



Area

Run time



Area

$$\text{Total Wasted Resources} = \sum (\text{Unused_Area}_i \times \text{Duration}_i)$$

Wasted resource for a partitioning

$$wr(P) = \sum_{i=1, \dots, k} wr(P_i)$$



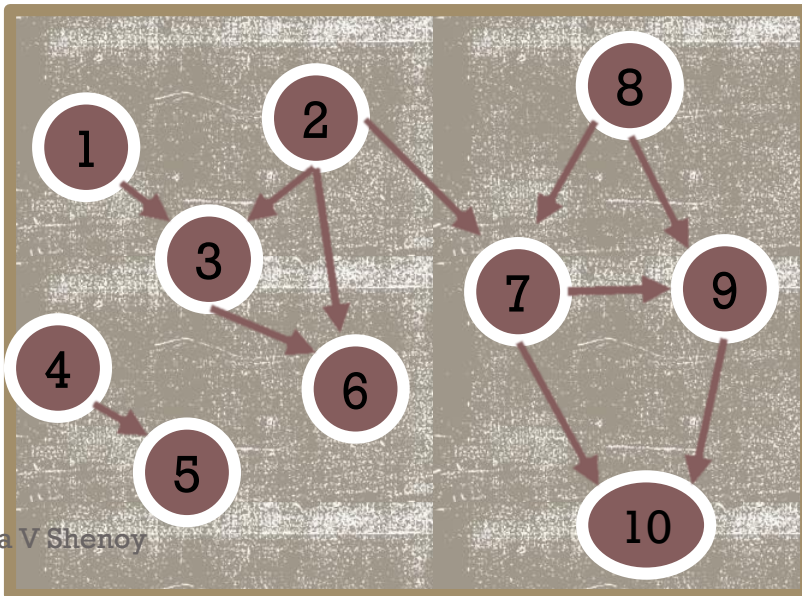
Communication Cost

Communication Cost: modelled as graph connectivity

Connectivity of a graph $G = (V, E)$:

$$con(G) = 2 * |E| / (|V|^2 - |V|)$$

$|V|^2 - |V|/2$: the number of all edges that can be built with V.



Connectivity
?

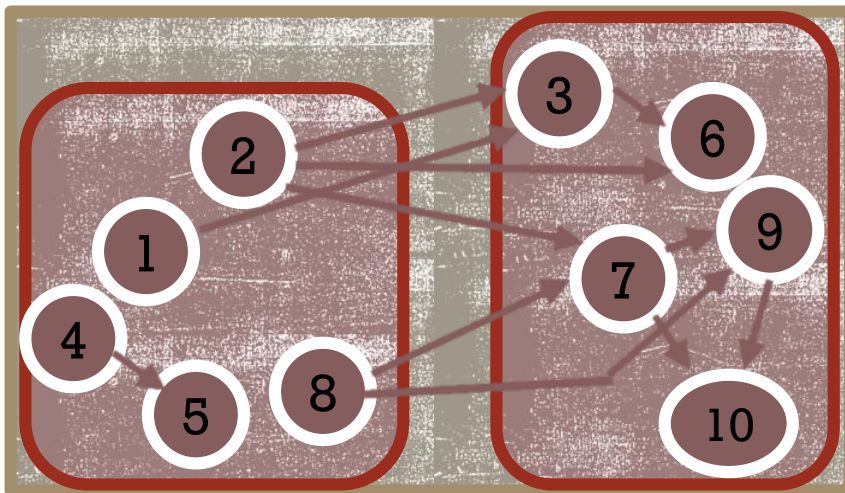
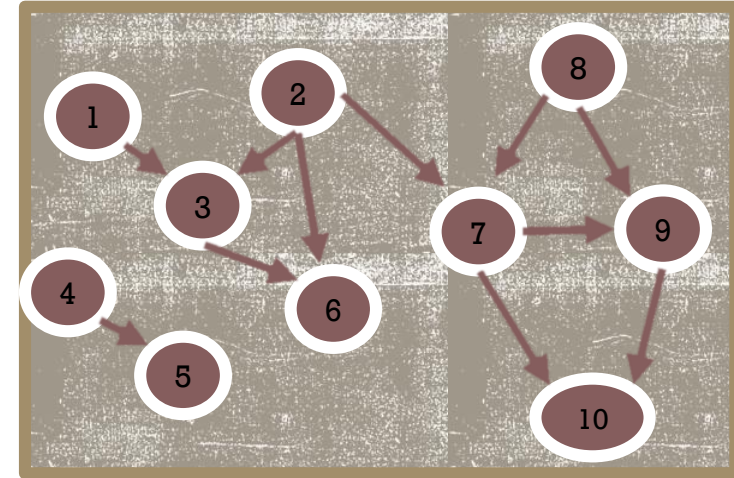


Communication overhead

Quality of Partitioning $P = \{P_1, \dots, P_n\}$

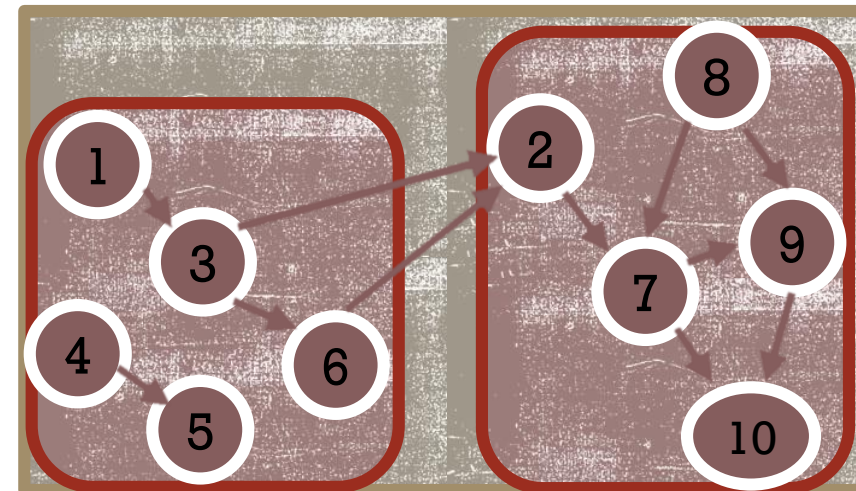
Average connectivity over P is called quality of the partition P

Assume each partition can have 5 nodes?



RC@Meetha V Shenoy

Quality = 0.25



Quality = 0.45



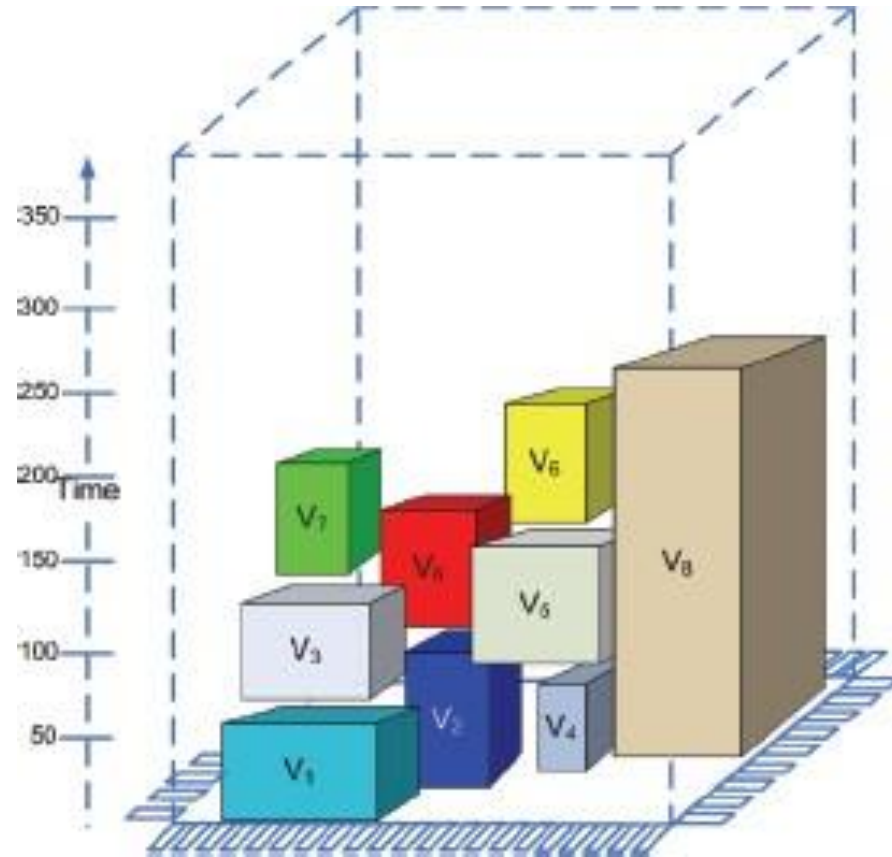
Communication overhead

Minimizing the number of crossing edges among the partitions.

Highly connected components are placed in the same partition (High quality partitioning)



TEMPORAL PLACEMENT



Host based configuration

Accelerator



TEMPORAL PLACEMENT

□ Off-line (compile time)

- Pre-defined placement sequence

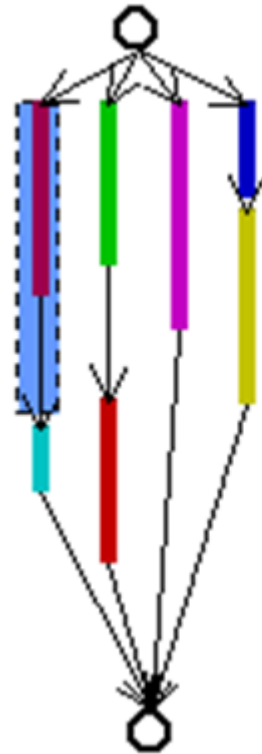
□ On-line (during execution)

- Computation sequence is not known in advance.
 - → Dynamically at run time.

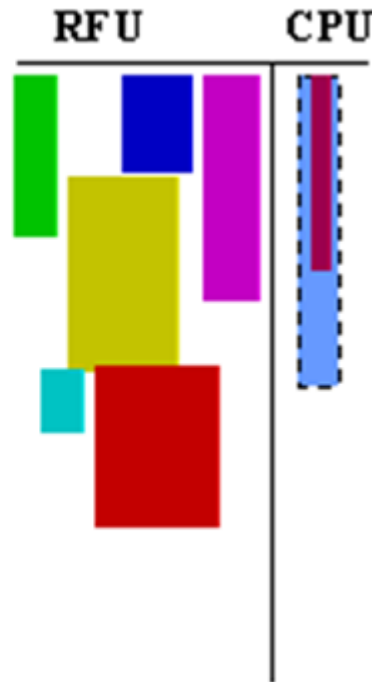


OFFLINE TEMPORAL PLACEMENT

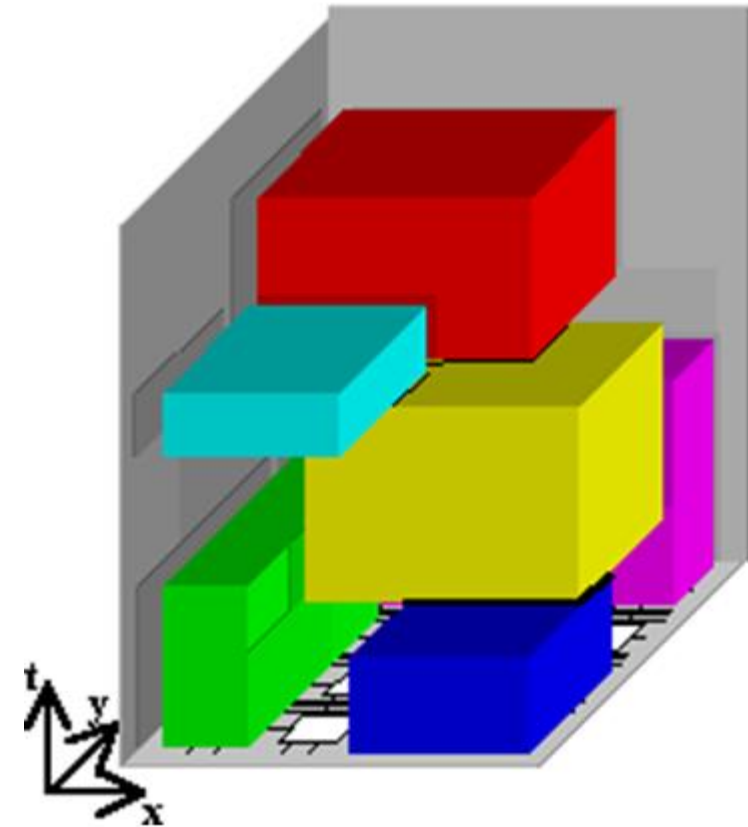
DFG



Schedule

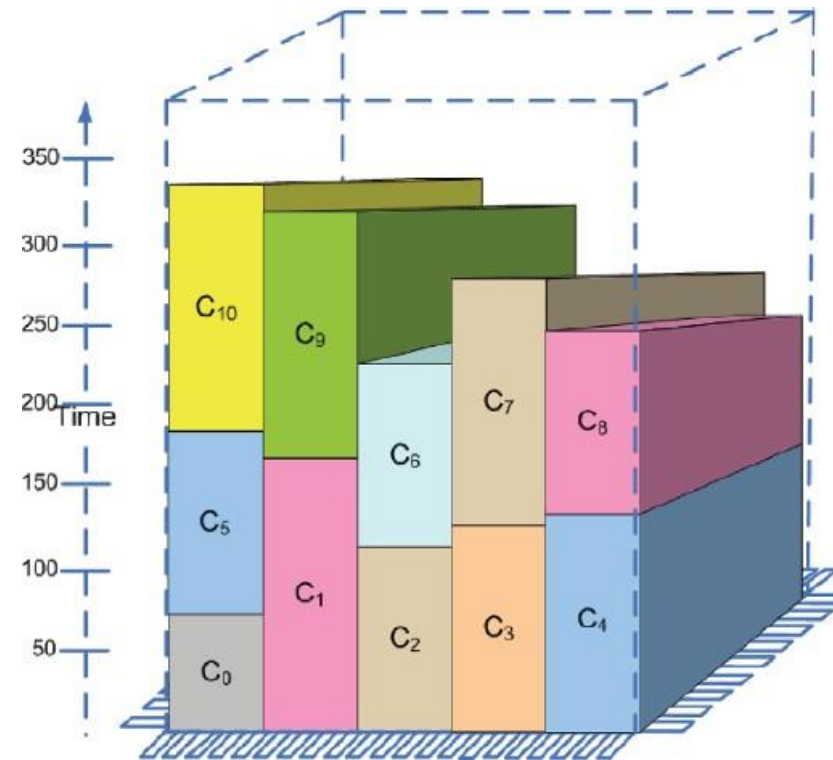


RFU



□ Configuration sequence produced:

- a)** {0, 1, 2, 3, 4}
- b)** {5, 1, 2, 3, 4}
- c)** {5, 1, 6, 7, 4}
- d)** {5, 1, 6, 7, 8}
- e)** {5, 9, 6, 7, 8}
- f)** {10, 9, 6, 7, 8}



TEMPORAL PLACEMENT ALGORITHMS

□ Off-Line temporal placement Algorithms:

1. First fit
2. Best fit
3. ILP

□ On-Line temporal placement Algorithms:

1. KAMER



BEST FIT

□ Algorithm: Best fit temporal placement of clusters

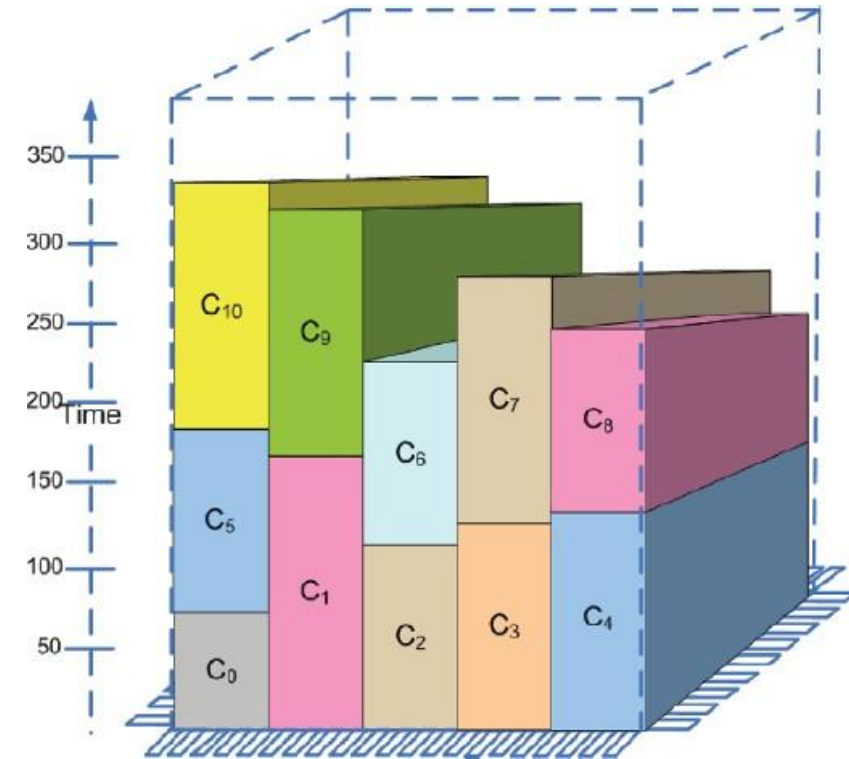
While all the clusters are not placed do

1. Select one cluster C_{act} from the list of ready-to-run clusters
2. From the clusters already placed, select the cluster C_{top} with the smallest finishing-time such that

$$C_{top} \leq C_{act}$$

3. Place the cluster C_{act} on top of C_{top}

end while



BEST FIT

□ Best fit:

1. Select a node among ready nodes.
2. Place it in the best free location.

□ Advantages:

- Better area utilization

□ Disadvantages:

- Much slower:
 - Complete list of free locations must be searched.
- Fragmentation



FIRST FIT

□ First fit:

1. Select a node among ready nodes.
 - i.e. nodes whose predecessors have been placed
2. Place it in the first free location.

□ Advantages:

- Fast (linear w.r.t. number of free locations)

□ Disadvantages:

- Unused resources very high
- Fragmentation



ILP

□ ILP:

- Can construct a constraint set
- An objective function

□ Advantage:

- Exact solution

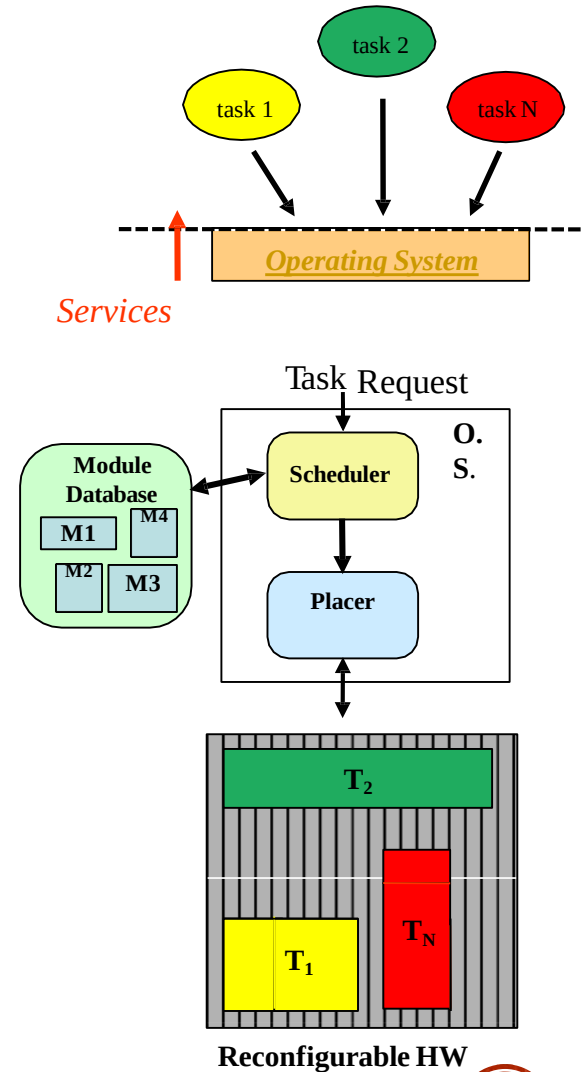
□ Limitation:

- Only for small size problems.



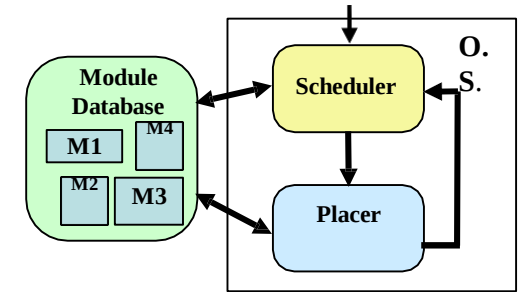
ONLINE PLACEMENT

- System Model: Operating system like
 - Scheduler: resource assignment to incoming tasks.
 - Module Database: Contain the physical implementation of the tasks.
 - Placer: Assign space on the reconfigurable device to run tasks sent by the scheduler

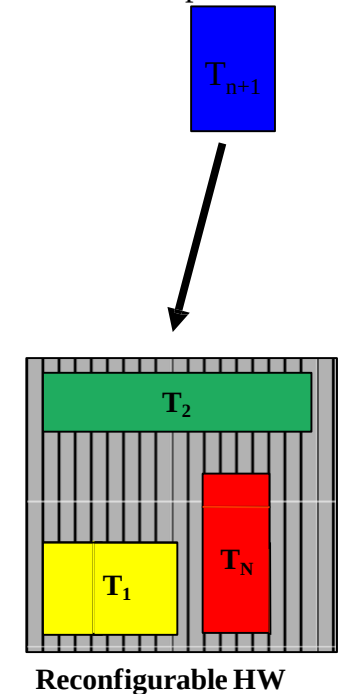


SCHEDULER PLACER INTERACTION

- The scheduler requests the placement of a new task.
- The placer try to find a place on the device for the new task.
 - On success, it acknowledges the scheduler and download the corresponding module to the device.
 - On failure it acknowledges the scheduler. The scheduler can now decided to reject the request or to try it later again.
- In all the case we have to decide **on line**, if a new component can be placed on a device where other component are already running



Is there a place for me ?



□ Larger picture:

- An RFU operation r_i can be either accepted or rejected
 - based on availability of RFU resources
- If rejected, run on CPU
 - → running time penalty

□ Assumption:

- After running an operation on RFU, results are saved in CPU registers



KAMER MODEL OF AN RCS

□ Empty Rectangle (ER):

- *A rectangle that does not overlap a placed module on the chip*

□ Maximum Empty Rectangle (MER):

- *An ER not included in any other ER than the device bounding box*



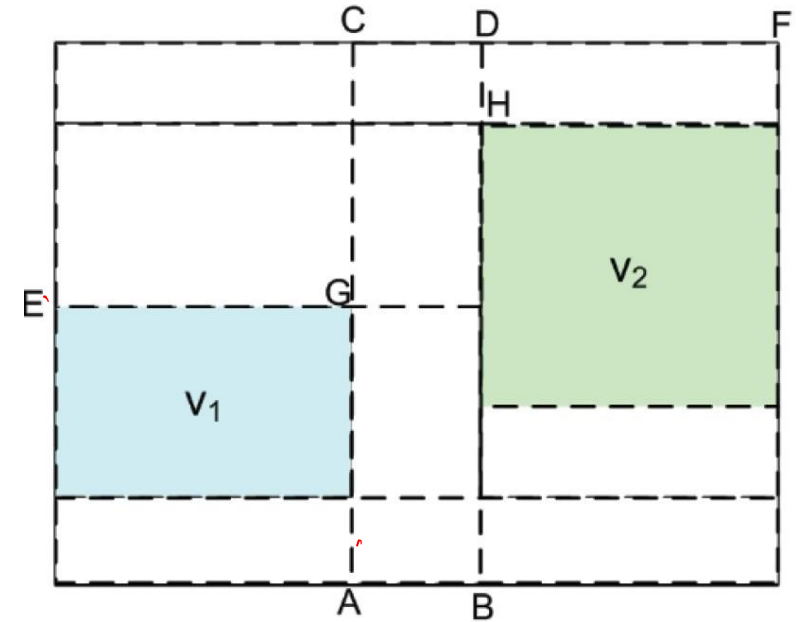
KAMER MODEL OF AN RCS

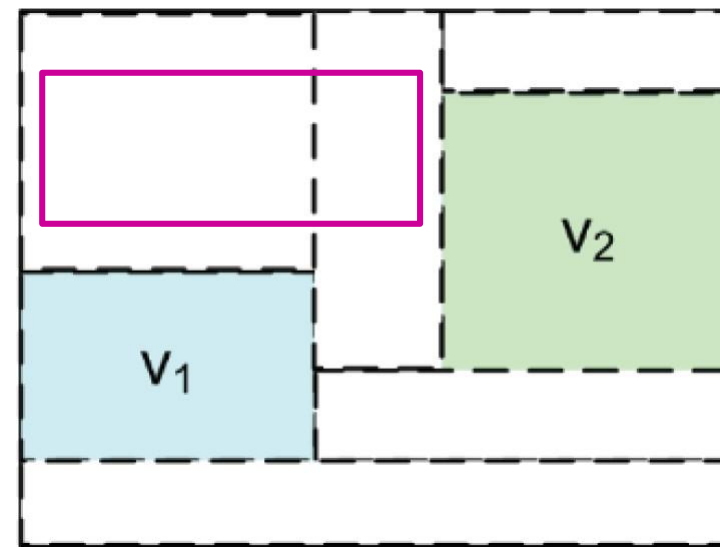
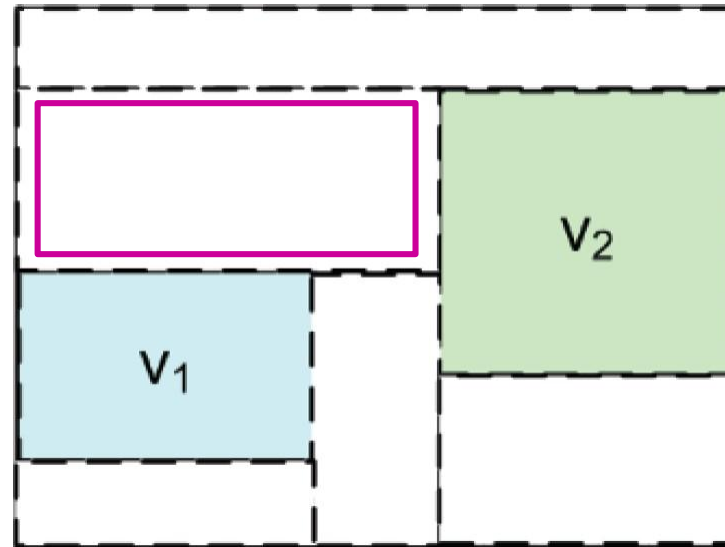
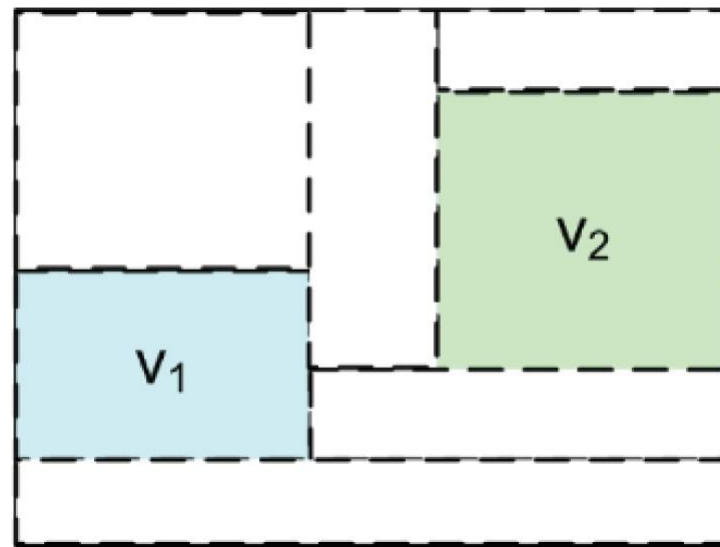
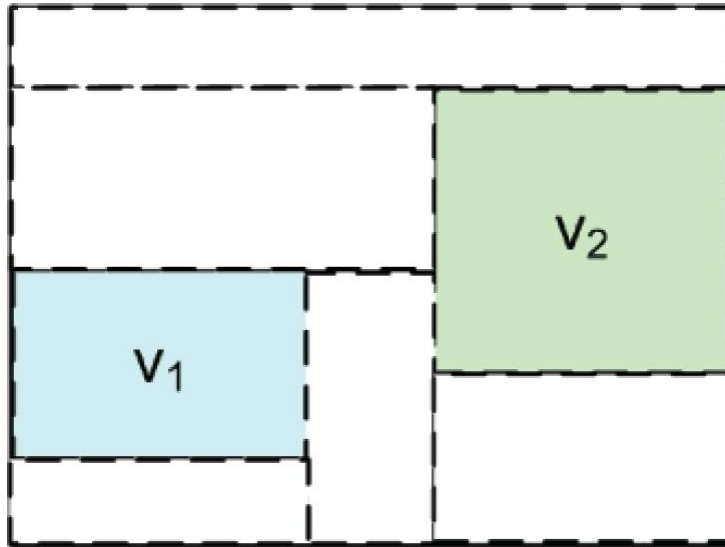
□ ER:

- (E,F) not ER
- (E,D): MER
- (A,D): MER
- (E,C): not MER

□ KAMER method:

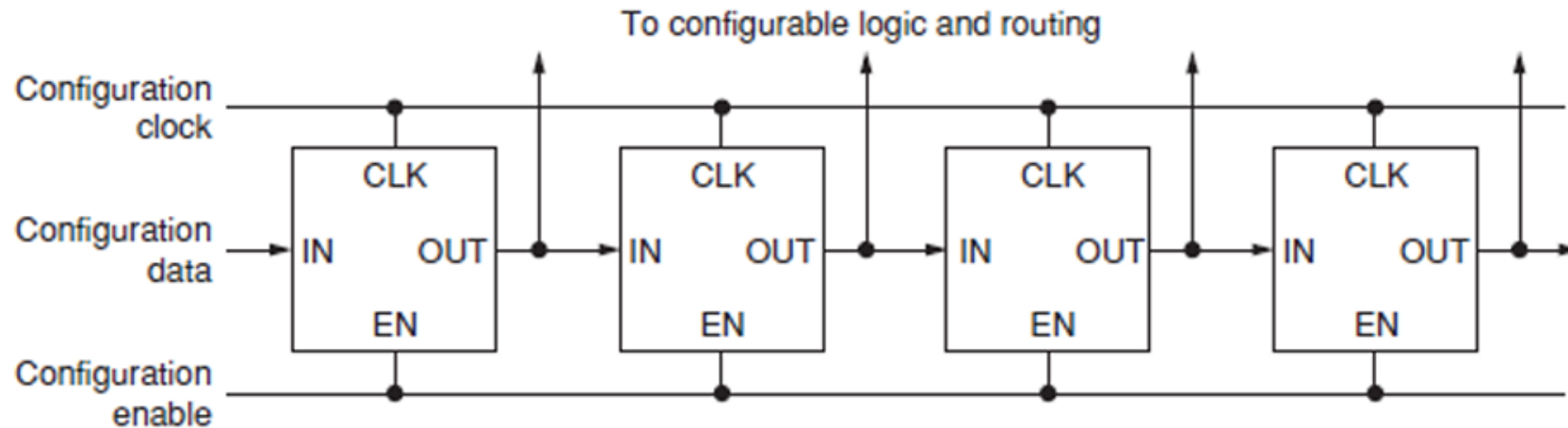
- *Keeping All Maximum Empty Rectangles:*
 - Permanently keeps track of all MERs
 - Whenever a request for placing a component v arrives, the list of MERs is searched for a rectangle that can accommodate v .
- Possible to have many MERs in which v can fit
- → Strategies: first-fit or best-fit
- Bottom left [Bazargan]





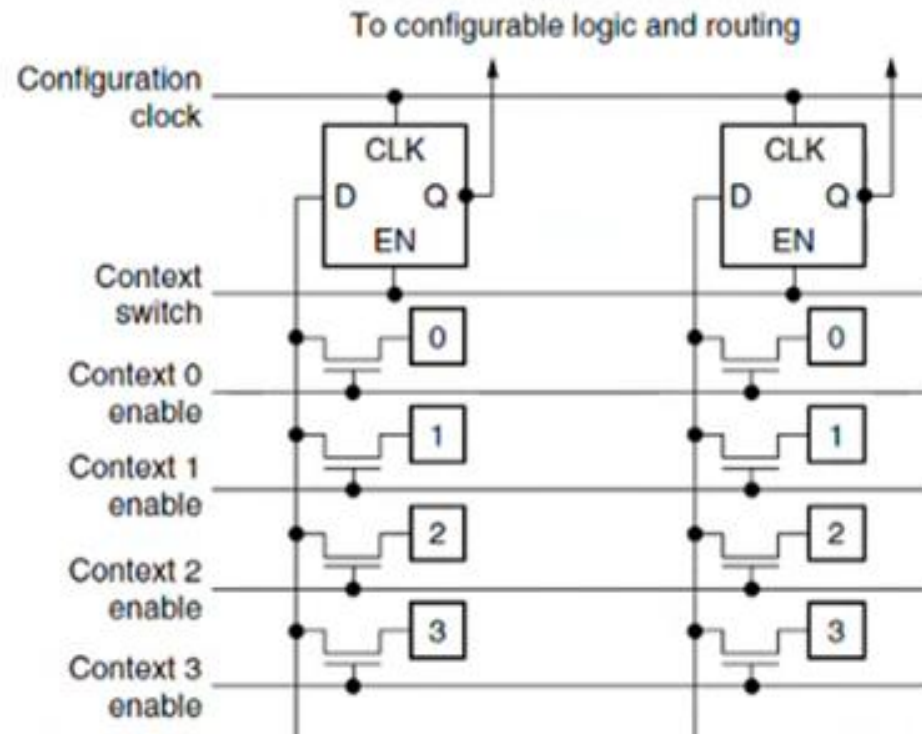
SINGLE-CONTEXT

- Serially programmed FPGAs shift in configuration data.
- o Each cell shown contains one SRAM bit of programming data.
- o The clock controls shifting during configuration



MULTI-CONTEXT FPGA

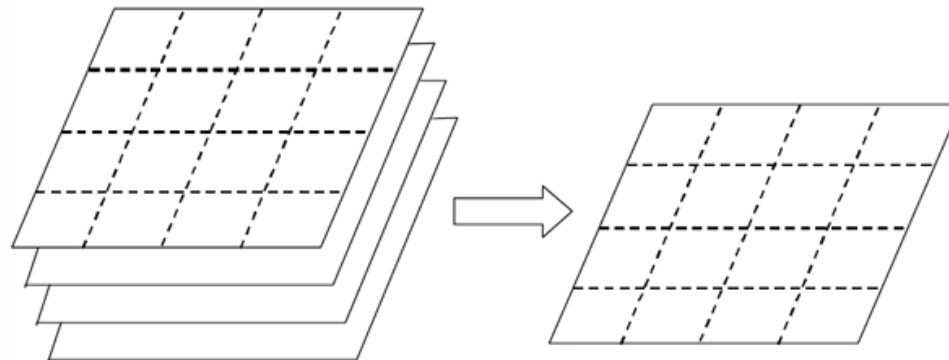
- Two multi-contexted configuration bits of a 4-context device.



MULTI-CONTEXT FPGA

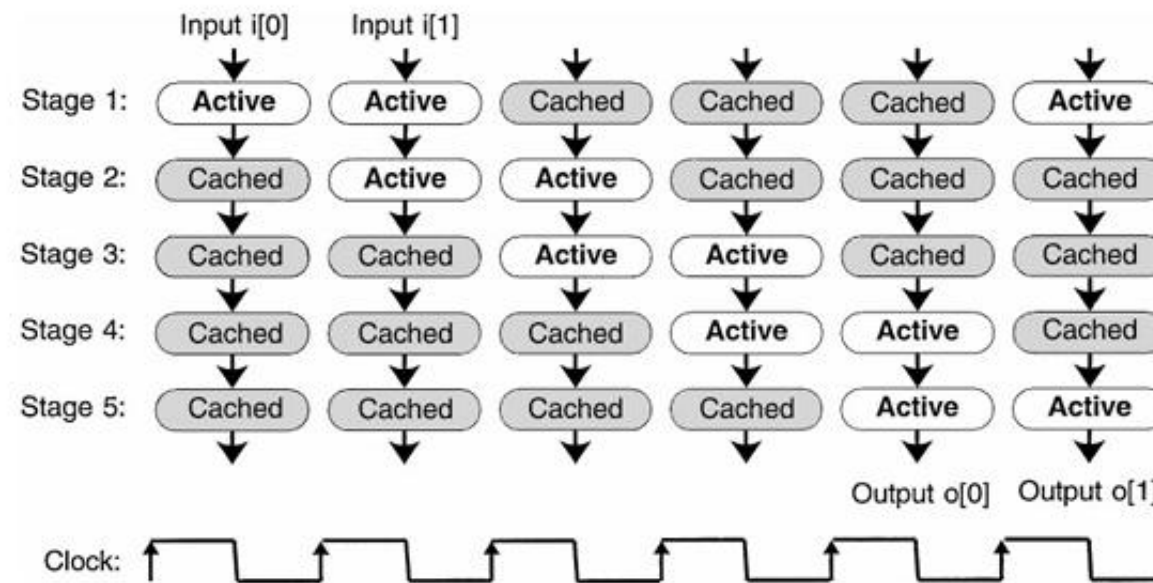
- Advantages:

- o Switch between stored configurations quickly (some in a single clock cycle)
 - Dramatically reducing reconfiguration overhead if the next configuration is present in one of the alternate contexts
- o Background loading of configuration data during circuit operation
 - Overlapping computation with reconfiguration



PIPELINE RECONFIGURATION

- FPGA hardware is dynamically reconfigured at each clock cycle to execute different processing stages.



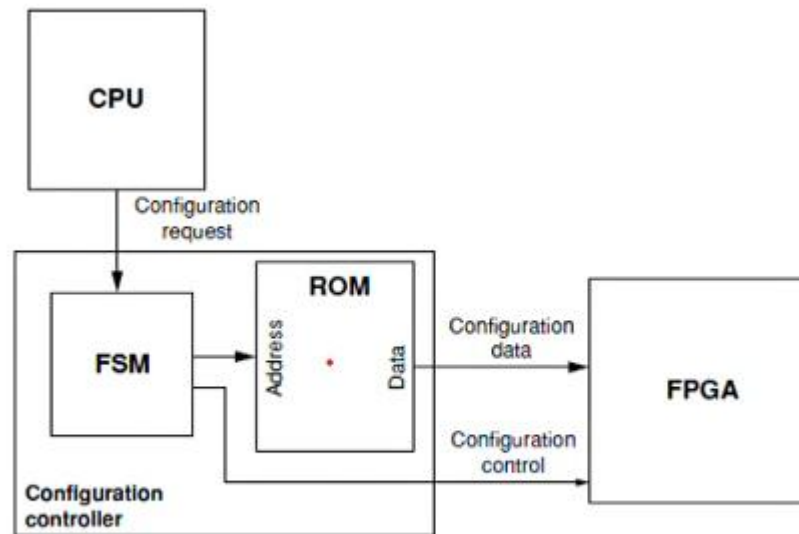
DESIGN CHALLENGES

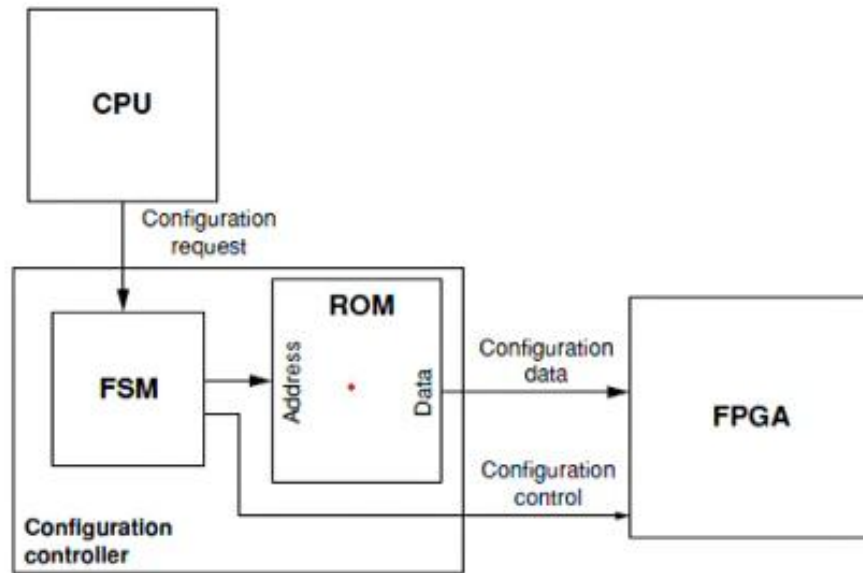
- Each clock cycle is divided into phases:
- **Configuration loading** (20% of cycle)
- **Data transfer** (15% of cycle)
- **Actual processing** (60% of cycle)
- **Cleanup/setup** (5% of cycle)



CONFIGURATION ARCHITECTURES

- Configuration data can be transferred to an FPGA by a specialized configuration controller containing non-volatile memory
- The reconfiguration process can be triggered by a CPU





- **Step 1: Configuration Request**
 - CPU determines which configuration is needed
 - Sends request to FSM with specific configuration ID
- **Step 2: FSM Control**
 - FSM translates CPU request into hardware actions
 - Sets ROM address to point to desired configuration
 - Sends control signals to prepare FPGA for new configuration
- **Step 3: Configuration Loading**
 - ROM outputs the bitstream data
 - FSM provides timing and control signals
 - FPGA receives and loads the new configuration
- **Step 4: Activation**



Design a **4-bit synchronous up/down counter** with the following specifications:

1.Inputs:

- clk : Clock signal
- rst : Synchronous reset (active high)
- up_down : Direction control (1 for up-count, 0 for down-count)
- enable : Enable counting

2.Outputs:

- count[3:0] : 4-bit counter value

3.Behavior:

- When rst is high, the counter resets to 4'b0000.
- When enable is high:
 - If up_down = 1, the counter increments on each rising clock edge.
 - If up_down = 0, the counter decrements on each rising clock edge.
- The counter should wrap around on overflow or underflow (i.e., 4'b1111 → 4'b0000 for up, 4'b0000 → 4'b1111 for down).

Write the Verilog code and the test bench for the same.

